

Chapter 10 Sets

The Set ADT (Abstract Data Type)

◆ Prompt 1: What is a Set ADT?

✓ Answer:

A Set ADT represents the **mathematical concept of a set**:

- A collection of **distinct elements**
- **No duplicates**
- **Unordered**

✦ Python implements this using:

- `set` → mutable
- `frozenset` → immutable

Both are implemented internally using **hash tables**.

◆ Prompt 2: What is the difference between `set` and `frozenset`?

✓ Answer:

Feature	<code>set</code>	<code>frozenset</code>
Mutable	✓ Yes	✗ No
Hashable	✗ No	✓ Yes
Can be modified	Yes	No
Underlying structure	Hash table	Hash table

✦ `frozenset` can be used as a key in dictionaries.

◆ Prompt 3: What abstract base classes correspond to sets?

✓ Answer:

Python's `collections` module provides:

- `collections.Set` → like `frozenset`

- `collections.MutableSet` → like `set`

✦ The **Set ADT** $\approx$ `MutableSet`

◆ Prompt 4: What are the five fundamental Set ADT operations?

✓ **Answer:**

Operation	Meaning
<code>S.add(e)</code>	Add element <code>e</code>
<code>S.discard(e)</code>	Remove <code>e</code> if present
<code>e in S</code>	Membership test
<code>len(S)</code>	Number of elements
<code>iter(S)</code>	Iterate through elements

✦ These five operations are sufficient to build all others.

◆ Prompt 5: What happens if we add a duplicate element?

✓ **Answer:**

Nothing happens.

```
S = {1, 2, 3}
```

```
S.add(2)
```

```
print(S)    # {1, 2, 3}
```

✦ Sets automatically prevent duplicates.

◆ Prompt 6: Difference between `discard` and `remove`?

✓ **Answer:**

Method	If element missing
<code>discard(e)</code>	No error
<code>remove(e)</code>	Raises <code>KeyError</code>

✦ Use `discard` when unsure.

◆ Prompt 7: What does `pop()` do in a set?

✓ Answer:

Removes and returns **any arbitrary element**.

★ Since sets are unordered, the element is unpredictable.

```
x = S.pop()
```

◆ Prompt 8: How does membership testing work (`e in s`)?

✓ Answer:

Uses **hashing** → average **O(1)** time.

★ This is one of the biggest advantages of sets.

◆ Prompt 9: Boolean set comparisons

✓ Answer:

Expression	Meaning
<code>S == T</code>	Same elements
<code>S != T</code>	Different elements
<code>S <= T</code>	Subset
<code>S < T</code>	Proper subset
<code>S >= T</code>	Superset
<code>S > T</code>	Proper superset
<code>S.isdisjoint(T)</code>	No common elements

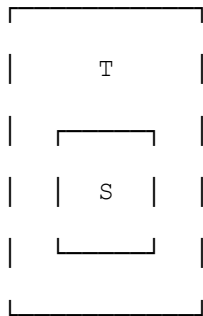
◆ Prompt 10: What is a subset?

✓ Answer:

Set `S` is a subset of `T` if:

Every element of `S` is in `T`

Diagram:



◆ Prompt 11: What is a proper subset?

✓ Answer:

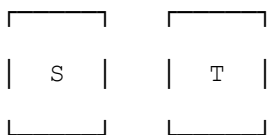
- $S \subset T$
- S is smaller than T
- They are **not** equal

◆ Prompt 12: What does `isdisjoint()` mean?

✓ Answer:

Returns `True` if two sets share **no** elements.

Diagram:



(no overlap)

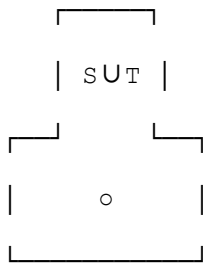
◆ Prompt 13: What is set union?

✓ Answer:

Union = all elements from both sets.

S | T

Diagram:



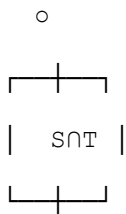
◆ Prompt 14: What is set intersection?

✓ Answer:

Intersection = common elements.

$S \cap T$

Diagram:



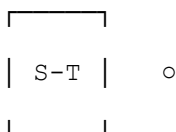
◆ Prompt 15: What is set difference?

✓ Answer:

Elements in S but not in T.

$S - T$

Diagram:



◆ Prompt 16: What is symmetric difference?

✓ **Answer:**

Elements in **exactly one** of the sets.

$$S \oplus T$$

◆ Prompt 17: Updating vs Non-updating operations

✓ **Answer:**

Operation	New set	Modify original
`S`	T`	✓
`S`	= T`	✗
S & T	✓	✗
S &= T	✗	✓

◆ Prompt 18: Why are sets implemented with hash tables?

✓ **Answer:**

Because they provide:

- Fast membership test $\rightarrow O(1)$
- Fast insert/delete
- Uniqueness guarantee